AWS Cloud Interview Guide

CLOUD WITH RAJ
www.cloudwithraj.com

Table of Contents

More To Be Added...

Raj's Bio

- Former Principal SA at aws
- Former Distinguished Cloud Architect at verizon
- 20+ Years of IT experience
- Designed and implemented multiple world-scale projects with official AWS blogs on them
- Bestselling author (60,000+ paid students)
- Presented highly-rated talks at major events
- LinkedIn Top Systems Design Voice

Lighthouse projects designed : Dr. B. Covid Vaccine Registration, Collins Flight Systems used by 100+ airlines, Freddie Mac Datacenter to AWS Migration, and more

Please follow me on socials:

Important AWS Services for Interviews

CLOUD WITH RAJ
www.cloudwithraj.com

Compute


EC2


Lambda


Elastic Kubernetes Service


ECS


Auto Scaling

Storage


S3


EBS


RDS


DynamoDB


ElastiCache

Network


VPC


Load Balancer


API Gateway


AWS Global Infrastructure (Region, AZ)

Security


KMS


IAM


WAF


Shield


GuardDuty


Secrets Manager


Config

Gen AI


BedRock


Q


SageMaker

Migration


DMS


Migration Hub


Application Migration Service


Application Discovery Service

Event Driven


SNS


SQS


EventBridge


Step Functions

Observability


CloudWatch


CloudTrail


X-Ray

Cost Optimization


Compute Optimizer


CloudWatch Insights


Cost Explorer


Budget

Spot Instance

Reserve Instance

Savings Plan

Analytics

Glue

EMR

Athena

QuickSight

Kinesis

Reporting

Redshift

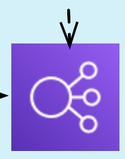
DevOps

CloudFormation

Microservices with ALB

CLOUD WITH RAJ
www.cloudwithraj.com

Domain: cloudwithraj.com



ALB
(url: cloudwithraj.com)
Path Based Routing

/browse
Target Group 1

/buy
Target Group 2

/* (Catch all)
Target Group 3

Auto Scaling Group

EC2
(Running Code)

EC2
(Scaled Up)

(Handles traffic of cloudwithraj.com/browse)

Lambda

(Handles traffic of cloudwithraj.com/buy)

EKS

(Handles traffic of anything else)

Database
Amazon Aurora

Database
Amazon DynamoDB

Database
Amazon Aurora

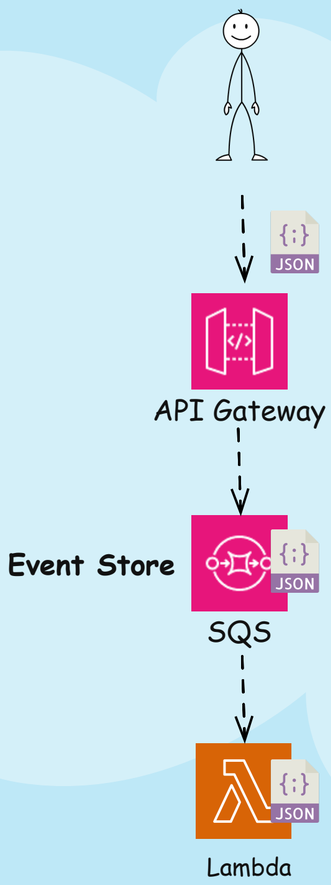
Microservice 1

Microservice 2

Microservice 3

Event Driven Architecture (Basic)

CLOUD WITH RAJ
www.cloudwithraj.com



An event-driven architecture decouples the producer and processor. In this example producer (human) invokes an API, and send information in JSON payload. API Gateway puts it into an event store (SQS), and the processor (Lambda) picks it up and processes it. Note that, the API gateway and Lambda can scale (and managed/deployed) independently

Benefits of an event-driven architecture

1. Scale and fail independently - By decoupling your services, they are only aware of the event router, not each other. This means that your services are interoperable, but if one service has a failure, the rest will keep running. The event router acts as an elastic buffer that will accommodate surges in workloads.

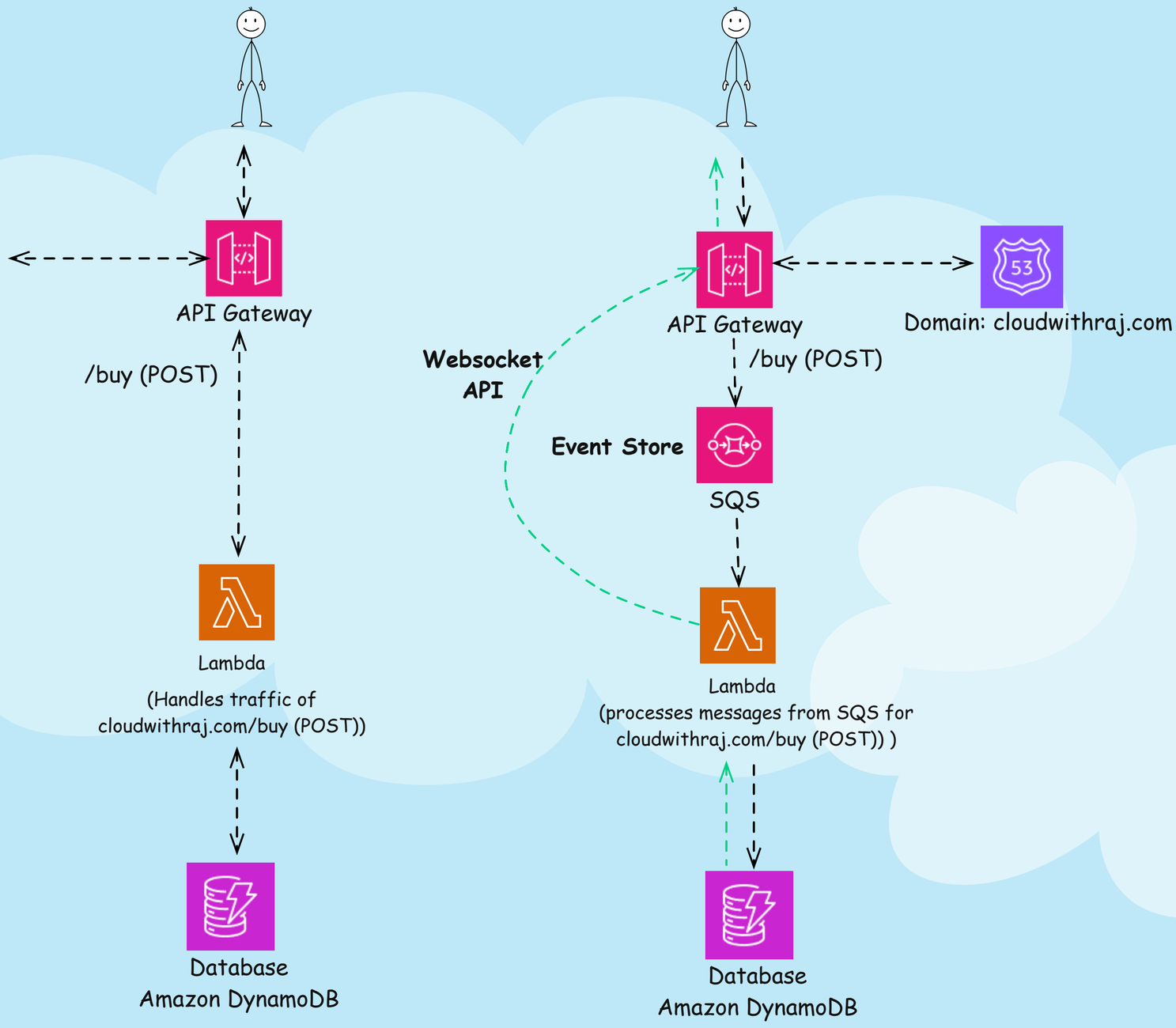
2. Develop with agility - You no longer need to write custom code to poll, filter, and route events; the event router will automatically filter and push events to consumers. The router also removes the need for heavy coordination between producer and consumer services, speeding up your development process.

3. Audit with ease - An event router acts as a centralized location to audit your application and define policies. These policies can restrict who can publish and subscribe to a router and control which users and resources have permission to access your data. You can also encrypt your events both in transit and at rest.

4. Cut costs - Event-driven architectures are push-based, so everything happens on-demand as the event presents itself in the router. This way, you're not paying for continuous polling to check for an event. This means less network bandwidth consumption, less CPU utilization, less idle fleet capacity, and less SSL/TLS handshakes.

Microservice Vs Event Driven

CLOUD WITH RAJ
www.cloudwithraj.com



Microservice

Microservice with Event Driven

The main differences are:

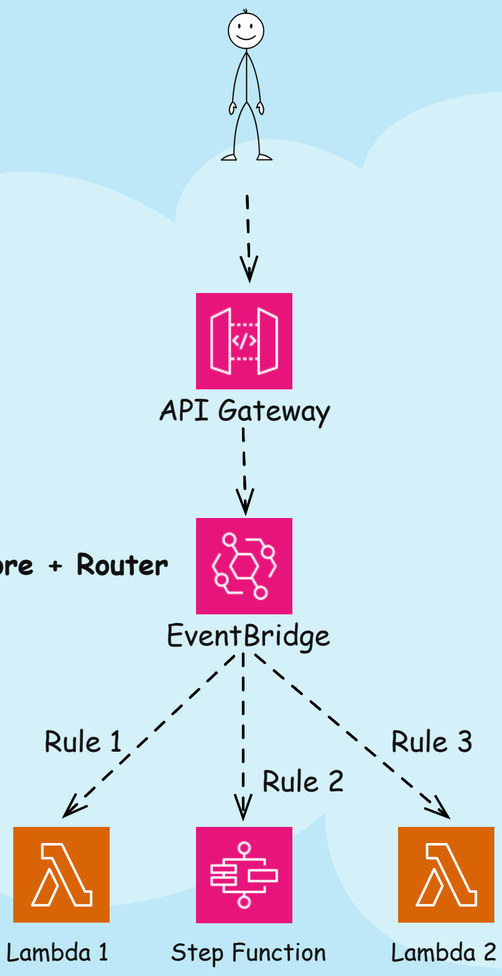
1. Traditional microservice is synchronous i.e. the request and response happens with the same invocation. Whereas with Event Driven, User gets a confirmation that message is inserted into SQS. But he doesn't get the response from the actual message processing by the Lambda in the same invocation. Instead the backend Lambda needs to send response out, in this case, using websocket APIs, to the user. Or the user can query the status afterwards

2. With EDA, API Gateway, and Lambda/Database can scale independently. Lambda can consume messages at a rate not to overwhelm the database

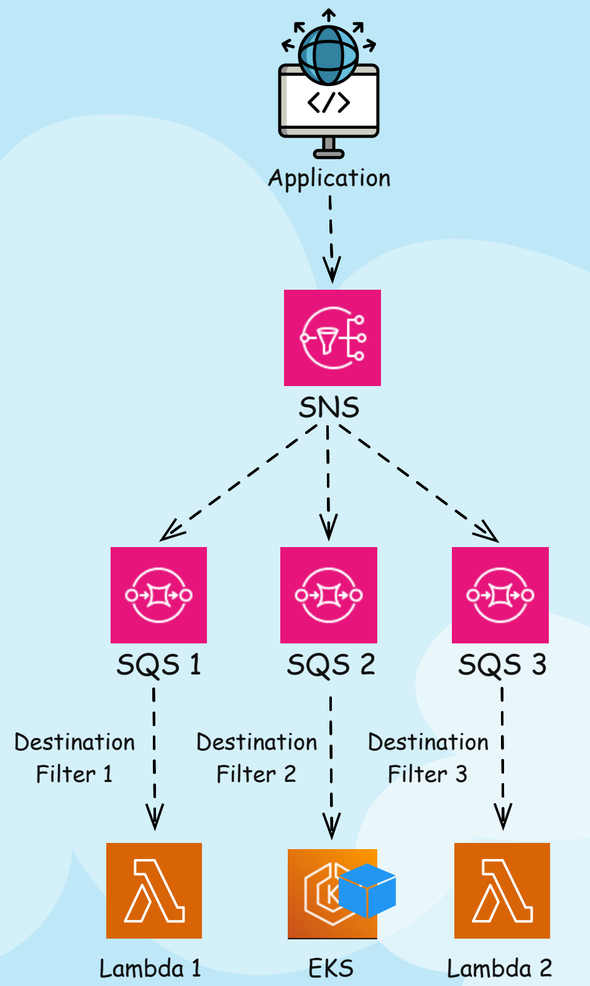
3. With EDA retries are built in. With microservices, if Lambda fails, user need to send the request again. With EDA, once the message is in SQS, even if Lambda fails, SQS will automatically retry

Event Driven Architecture (Advanced)

CLOUD WITH RAJ
www.cloudwithraj.com



Based on values in the message, EventBridge can fire different targets

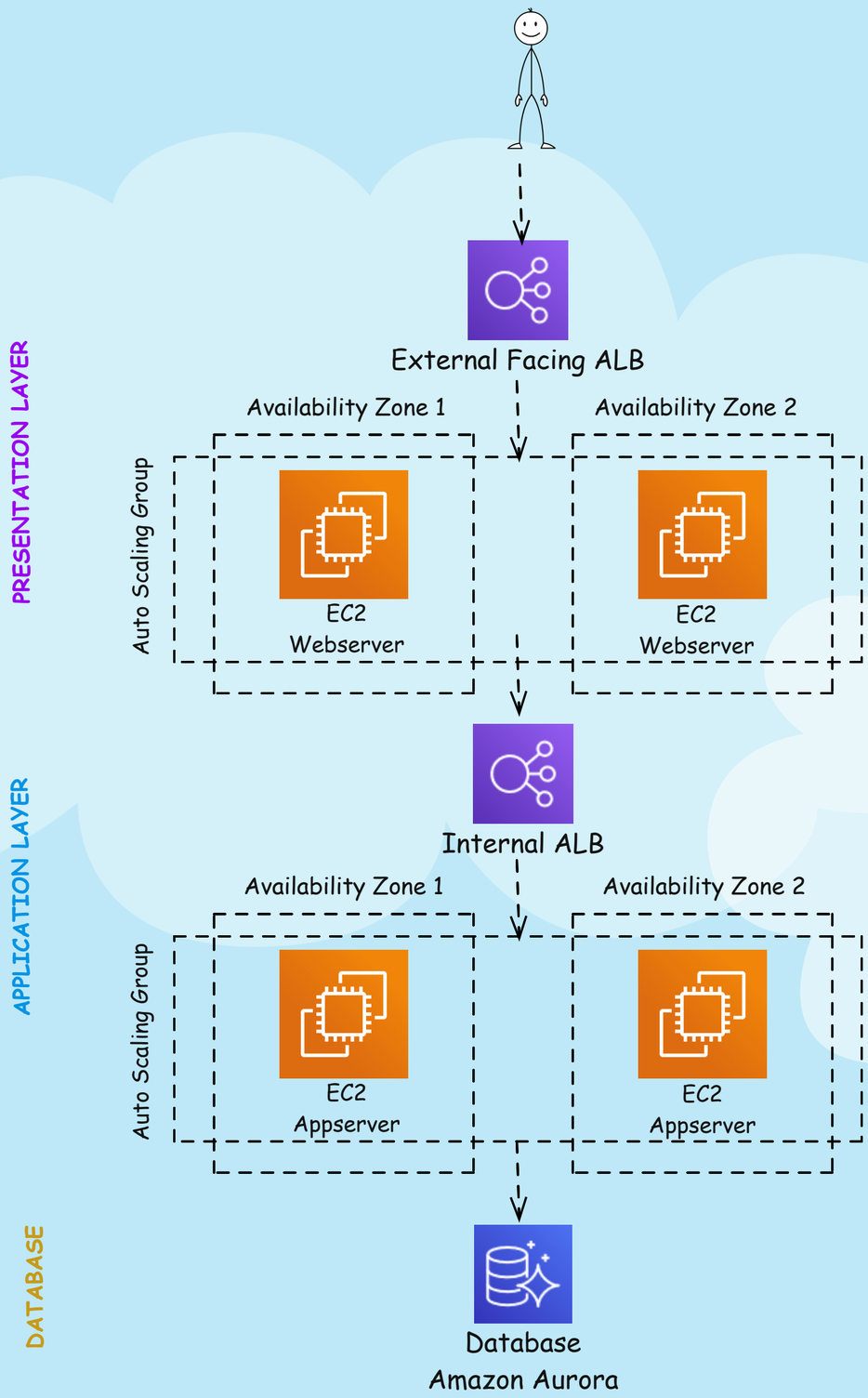


Based on values in the message, SNS can fire different targets

SNS Vs SQS Vs EventBridge Detailed Video:

3 Tier Architecture

CLOUD WITH RAJ
www.cloudwithraj.com



1. First layer is presentation layer. Customers consume the application using this layer. Generally, this is where the front end runs. For example - amazon.com website. This is implemented using an external facing load balancer distributing traffic to VMs (EC2s) running webserver.

2. Second layer is application layer. This is where the business logic resides. Going with the previous example - you browsed your products on amazon.com, and now you found the product you like and then click "add to cart". The flow comes to the application layer, validates the availability, and then creates a cart. This layer is implemented with internal facing load balancer and VMs running applications.

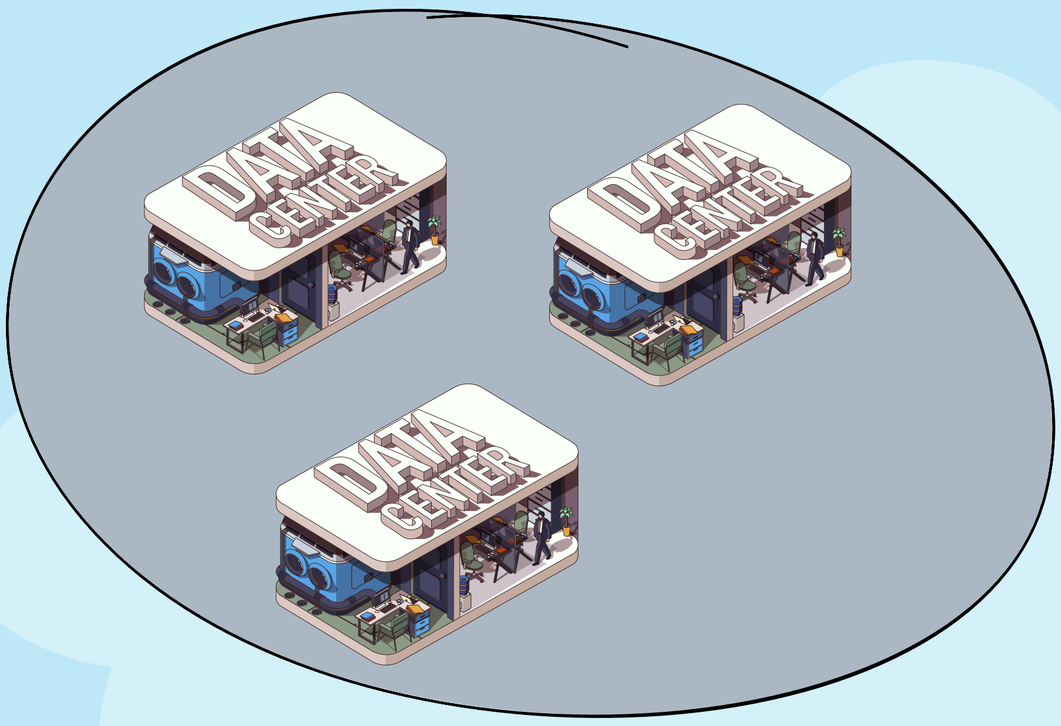
3. The last layer is the database layer. This is where information is stored. All the product information, your shopping cart, order history etc. Application layer interacts with this layer for CRUD (Create, Read, Update, Delete) operations. This could be implemented using one or mix of databases - SQL (e.g. Amazon Aurora), and/or NoSQL (DynamoDB)

Lastly - why is this so popular in interviews? This architecture comprised of many critical patterns - microservices, load balancing, scaling, performance optimization, high availability, and more. Based on your answers, interviewer can dig deep and check your understanding of the core concepts.

How Many Data Centers in One Availability Zone?

CLOUD WITH RAJ
www.cloudwithraj.com

Incorrect Answer:
One Availability Zone means one data center



1 Availability Zone

Correct Answer:
An AWS availability zone (AZ) can contain multiple data centers. Each zone is usually backed by one or more physical data centers, with the largest backed by as many as five.

Is AWS Lambda Cheaper than Amazon EC2?

CLOUD WITH RAJ
www.cloudwithraj.com

Incorrect Answer:
Yes, AWS Lambda is cheaper than Amazon EC2

s

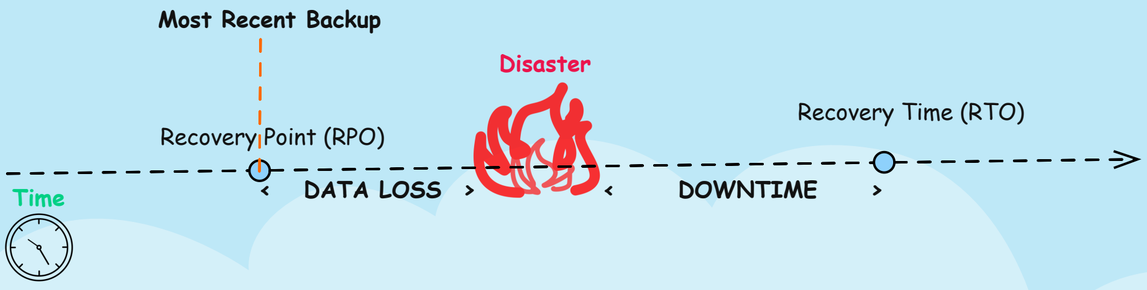
Lambda Cost Factors:
Architecture (x86 vs. Graviton)
Number of requests
Duration of each request
Amount of memory allocated (NOT used)
Amount of ephemeral storage allocated

EC2 Main Cost Factors:
Instance family
Attached EBS
Duration of EC2 runtime

Correct Answer:
It depends on the application. Both Lambda and EC2 have different cost factors (see above).
It is possible that, depending on the application, AWS Lambda can have a higher charge than EC2, and vice versa. it is important to consider not just the compute cost but the TCO (Total Cost of Ownership). With AWS Lambda there is no AMI to maintain, patch, and rehydrate, reducing management overhead and hence overall TCO.

RPO Vs RTO

CLOUD WITH RAJ
www.cloudwithraj.com

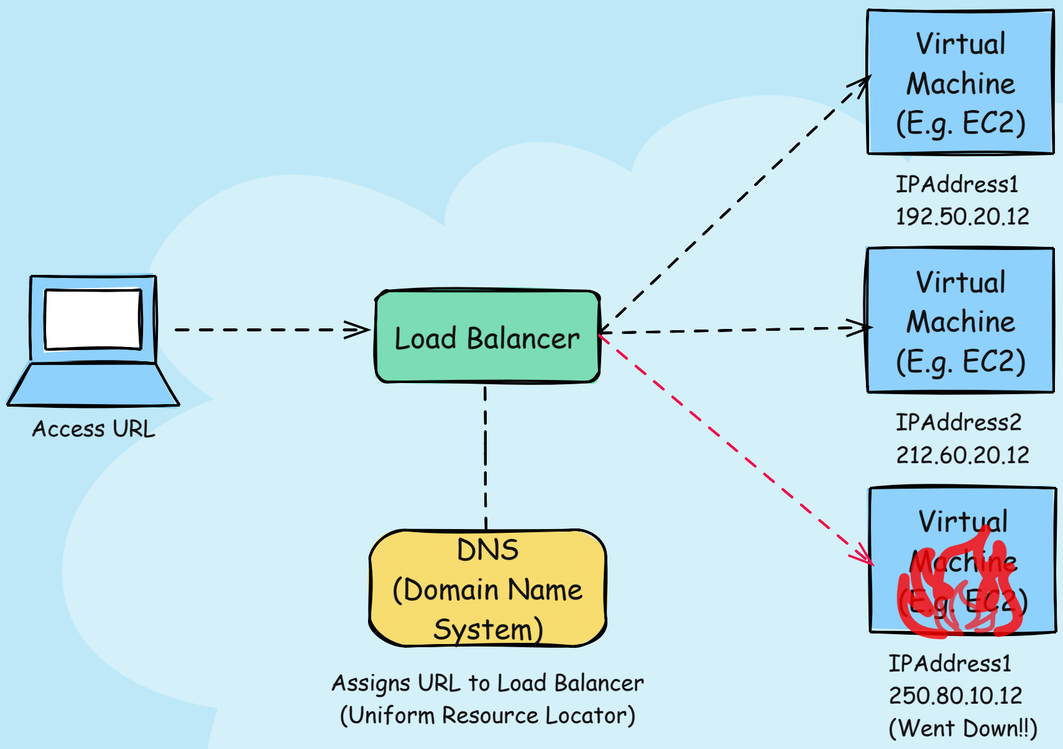


Candidates are sometimes confused by RPO thinking it's measured in unit of data, e.g. gigabyte, petabyte etc.

Correct Answer:
Both RPO and RTO are measured in time. RTO stands for Recovery Time Objective and is a measure of how quickly after an outage an application must be available again. RPO, or Recovery Point Objective, refers to how much data loss your application can tolerate. Another way to think about RPO is how old can the data be when this application is recovered i.e. the time between the last backup and the disaster. With both RTO and RPO, the targets are measured in hours, minutes, or seconds, with lower numbers representing less downtime or less data loss. RPO and RTO can be and often have different values for an application.

IP Address Vs. URL

CLOUD WITH RAJ
www.cloudwithraj.com



Bad Answer:
URL is a link assigned to an IP address

Correct Answer:
IP Address is a unique number that identifies a device connected to the internet, such as a Virtual Machine running your application. However, accessing a resource using this unique number is cumbersome; moreover, let's say when a VM comes down (the bottom one in the diagram), a new VM comes up to replace it with a different IP address. Hence, in reality, application running inside the VM is accessed using URL or Uniform Resource Locator.

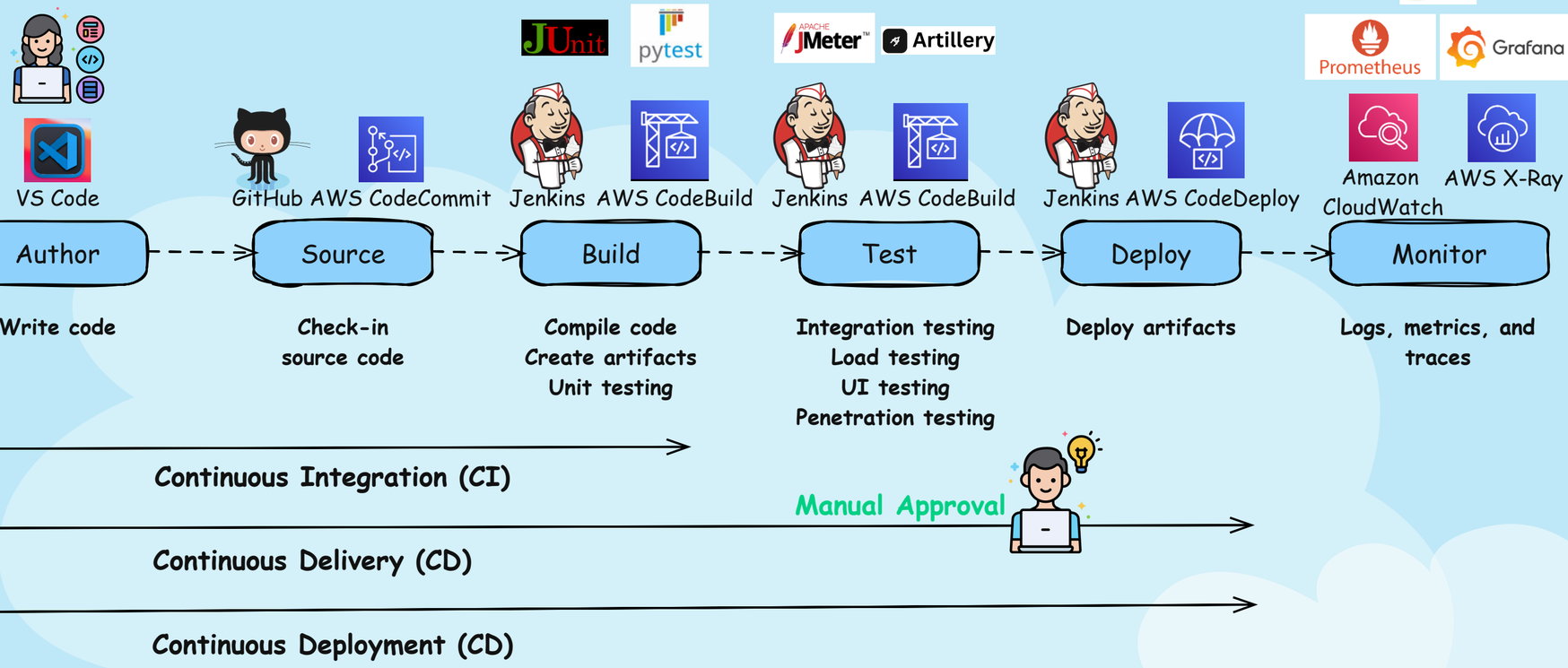
One URL does generally NOT map to one IP address; rather, the URL (e.g., www.amazon.com) is mapped to a Load Balancer, and that Load Balancer distributes traffic to multiple VMs with different IP addresses. Even if one VM goes down and another comes up, this Load Balancer using a URL always works because the Load Balancer appropriately distributes traffic across healthy instances. This way, you (the user) do not need to worry about the underlying IP addresses.

DevOps CICD Phases with Tools

CLOUD WITH RAJ
www.cloudwithraj.com

DevOps CICD Phases

CLOUD WITH RAJ
www.cloudwithraj.com



Kubernetes Tools Ecosystem with AWS

CLOUD WITH RAJ
www.cloudwithraj.com

Cloud Implementation

Amazon EKS

Observability

Prometheus

Grafana

Fluentbit

Jaeger

ADOT

CloudWatch

X-Ray

Scaling

Karpenter

AutoScaling

KEDA

Delivery/Automation

Argo

Terraform

Jenkins

Github Actions

Gitlab CICD

Security

Gatekeeper

Trivvy

ECR Scan

GuardDuty

Kube Bench

Secrets Manager

Istio

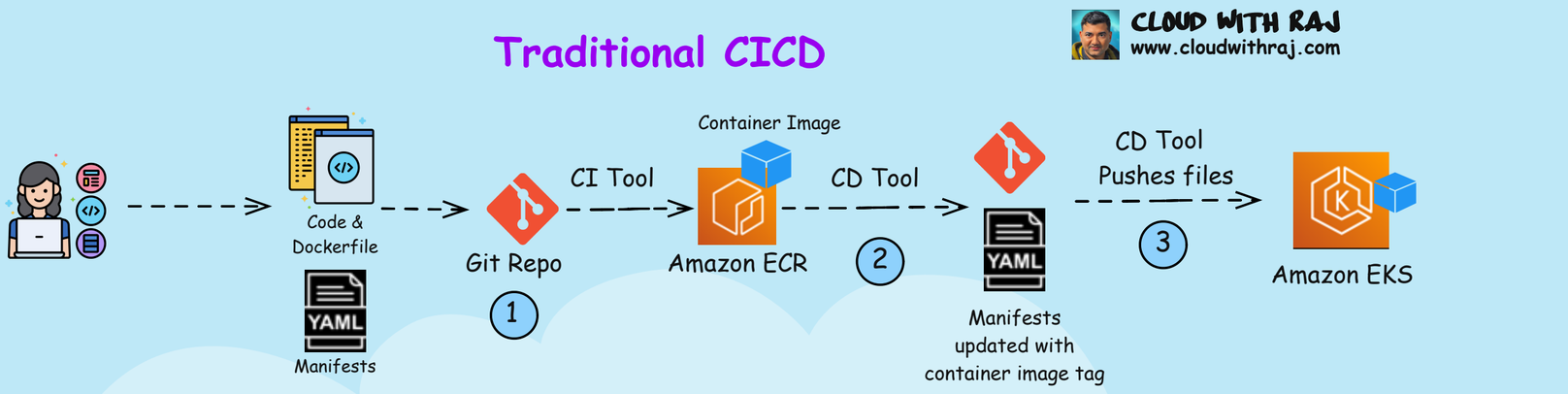
Cost Optimization

CloudWatch Container Insights

Cost and Usage Report (New feature - Split Cost Allocation)

Kubecost

Traditional CICD Vs. GitOps



Traditional DevOps

Step 1: Developers check in Code, Dockerfile, and manifest YAMLs to an application repository. CI tools (e.g., Jenkins) kick off, build the container image and save the image in a container registry such as Amazon ECR.

Step 2: CD tools (e.g. Jenkins) update the deployment manifest files with the tag of the container image.

Step 3: CD tools (e.g. Jenkins) execute the command to deploy the manifest files into the cluster, which, in terms, deploys the newly built container in the Amazon EKS cluster.

Conclusion - Traditional CICD is a push based model. If a sneaky SRE changes the YAML file directly in the cluster (e.g. changes number of replica, or even the container image itself!), the resources running in the cluster will deviate from what's defined in the YAML in the Git. Worse case, this change can break something, and DevOps team need to rerun part of the CICD process to push the intended YAMLs to the cluster

GitOps

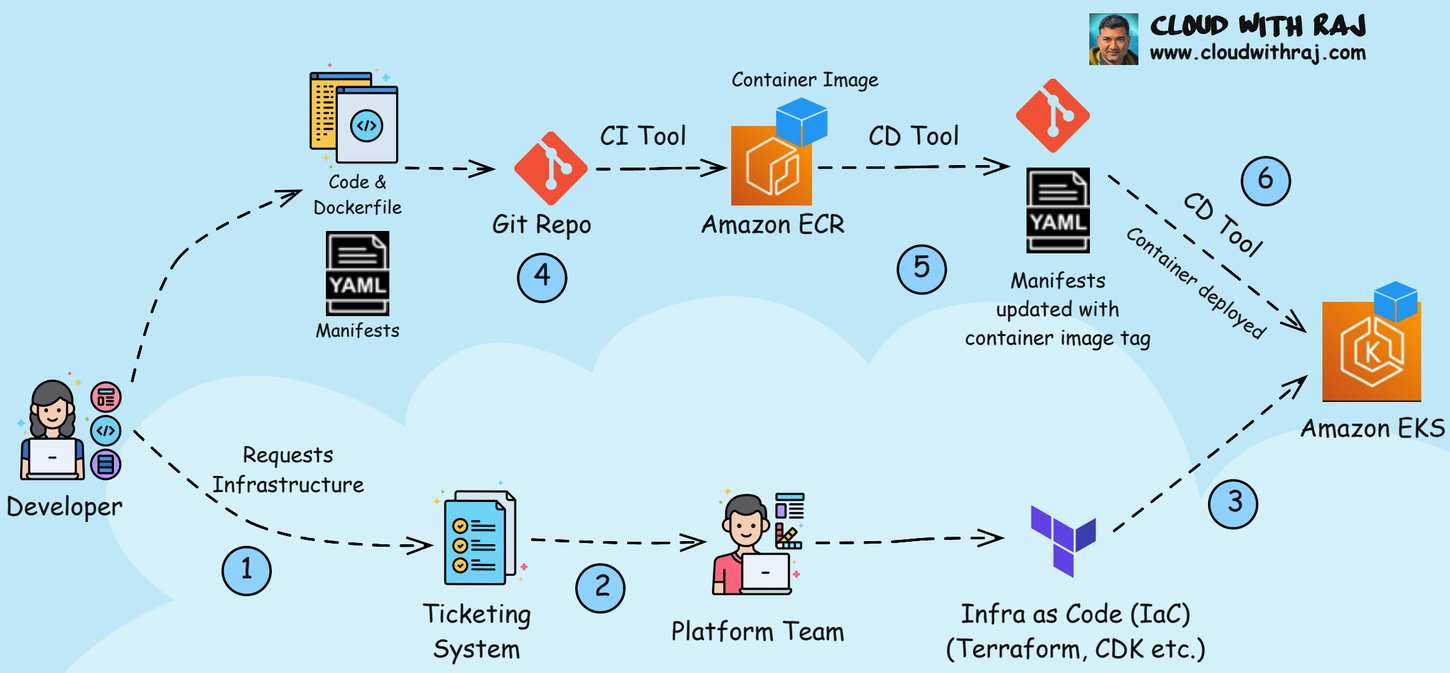
Step A: Developers check in Code, Dockerfile, and manifest YAMLs to an application repository. CI tools (e.g., Jenkins) kick off, build the container image and save the image in a container registry such as Amazon ECR.

Step B: CD tools (e.g. Jenkins) update the deployment manifest files with the tag of the container image.

Step C: With GitOps, Git becomes the single source of truth. You need to install a GitOps tool like Argo inside the cluster and point to a Git repo. Git keeps checking if there is a new file, or if the files in the cluster drifts from the ones in Git. As soon as YAML is updated with new container image, there is a drift between what's running in the cluster vs what's in Git. ArgoCD pulls in this updated YAML file and deploys new container.

Conclusion - GitOps does NOT replace DevOps. As you can see GitOps only replaces part of the CD process. If we think about the previous scenario where the sneaky SRE directly changes the YAML in cluster, ArgoCD will detect the mismatch between the changed file vs the one in Git. Since there is a difference, it will pull in the file from Git and bring Kubernetes resources to it's intended state. And don't worry, Argo can also send a message to the sneaky SRE's manager ;).

Platform Team and Developer Team



Recently, the term "platform team" has been floating around plenty. But what do platform team do? How are they different from the developer team? Let's understand with the diagram below:

Step 1: The developer team requests the Platform team to provision appropriate AWS resources. In this example, we are using Amazon EKS for the application, but this concept can be extended to any other AWS service. This request for AWS resources is typically done via the ticketing system.

Step 2: The platform team receives the request.

Step 3: The platform team uses Infrastructure as Code (IaC), such as Terraform, CDK, etc., to provision the requested AWS resources, and share the credentials with the Developer team.

Step 4: The developer team kicks off the CICD process. We are using a container process to understand the flow. Developers check in Code, Dockerfile, and manifest YAMLs to an application repository. CI tools (e.g., Jenkins, GitHub actions) kick off, build the container image and save the image in a container registry such as Amazon ECR.

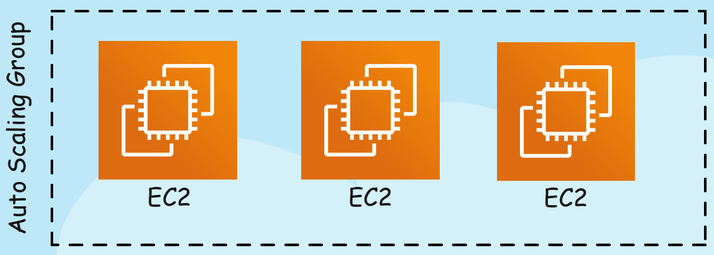
Step 5: CD tools (e.g. Jenkins, Spinnaker) update the deployment manifest files with the tag of the container image.

Step 6: CD tools execute the command to deploy the manifest files into the cluster, which, in terms, deploys the newly built container in the Amazon EKS cluster.

Conclusion - The platform team takes care of the infrastructure (often with the guardrails) appropriate for the organization, and the developer team uses that infrastructure to deploy their application. The platform team does the upgrade and maintenance of the infrastructure to reduce the burden on the developer team.

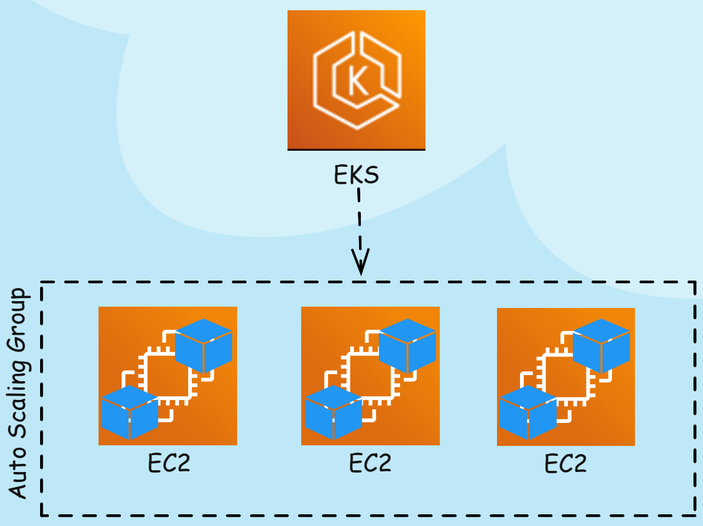
Scaling Difference Between Lambda, EC2, EKS

CLOUD WITH RAJ
www.cloudwithraj.com



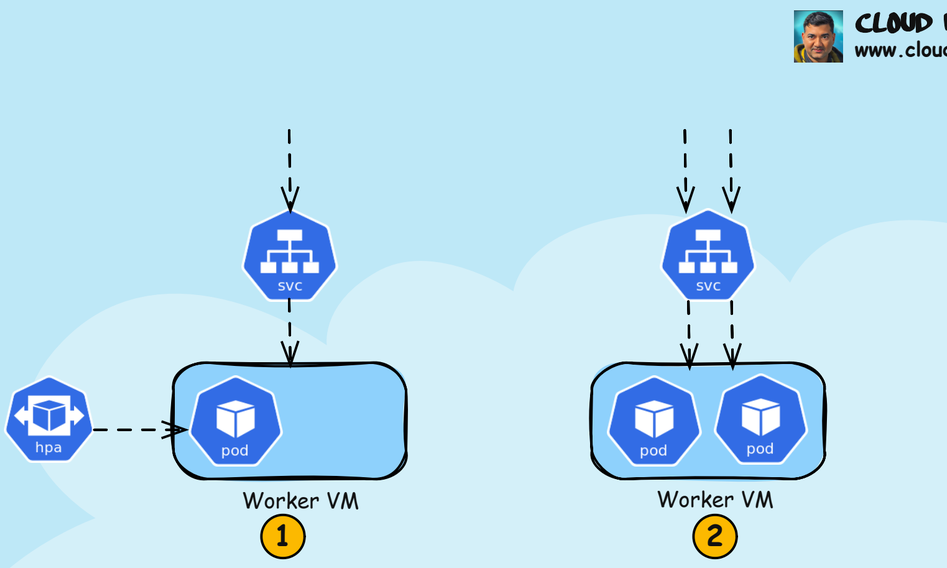
EC2 Scaling: You need to use a Auto Scaling Group (ASG) and define on what EC2 metric you want it to scale e.g. CPU utilization. You can use ASG's "minimum number of instances" to run certain number of instances on all time. Recently ASG also supports scheduled scaling, and warm pool.

Lambda Scaling: No ASG needed. For each incoming connection, Lambda automatically scales. Consider increasing the concurrency setting for Lambda as needed. Implement Provisioned Concurrency to keep certain number of Lambda pre-warmed. This can be done either with schedule or based on Provisioned Concurrency utilization.



EKS Scaling: EKS scaling is the most complex scaling. You may or may NOT use a Auto Scaling Group but it does NOT work like regular EC2 scaling. Please refer to the next page to learn about EKS (Kubernetes/K8s) scaling in detail

How Does Kubernetes Worker Nodes Scale?



Incorrect Answer:
Set Auto Scaling Groups to scale at a certain VM metric utilization like scaling regular VMs.

Correct Answer:
Step 1: You configure HPA (Horizontal Pod Autoscaler) to increase the replica of your pods at a certain CPU/Memory/Custom Metrics threshold.

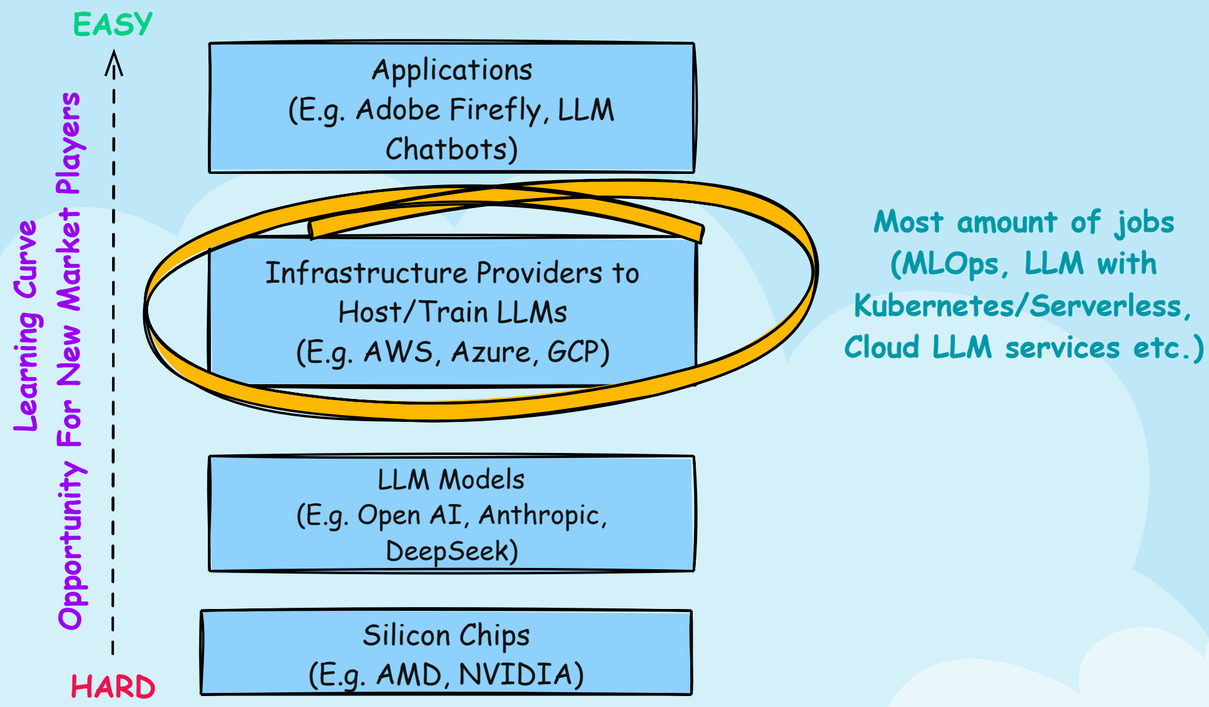
Step 2: As traffic increases and the pod metric utilization crosses the threshold, HPA increases the number of pods. If there is capacity in the existing worker VMs, then the Kubernetes kube-scheduler binds that pod into the running VMs.

Step 3: Traffic keeps increasing, and HPA increases the number of replicas of the pod. But now, there is no capacity left in the running VMs, so the kube-scheduler can't schedule the pod(yet!). That pod goes into pending, unschedulable state

Step 4: As soon as pod(s) go to pending unschedulable state, Kubernetes node scalers (such as Cluster Autoscaler, Karpenter etc.) provisions a new node. Cluster Autoscaler requires an Auto Scaling Group where it increases the desired VM count, whereas Karpenter doesn't require an Auto Scaling Group or Node Group. Once the new VM comes up, kube-scheduler puts that pending pod into the new node.

Gen AI 4 Layers

CLOUD WITH RAJ
www.cloudwithraj.com



Gen AI hype is at an all-time high, and so is the confusion. What do you study, how do you think about it, and where are the most jobs? These are the burning questions in our minds. Gen AI can be broken down into the following four layers.

1. The bottom layer is the hardware layer, i.e., the silicon chips that can train the models. Example - AMD, NVIDIA

2. Then comes the LLM models that get trained and run on the chips. Examples are Open AI, Anthropic etc.

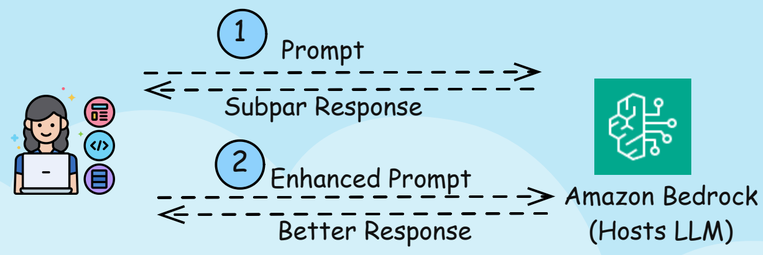
3. Then comes infrastructure providers who provide an easier way to consume, host, train, inference the models. Example is AWS. This layer consists of managed services such as Amazon Bedrock, which hosts pre-trained models, or provision VMs (Amazon EC2) where you can train your own LLM

4. Finally, we have the application layer which uses those LLMs. Some examples are Adobe Firefly, LLM chatbots, LLM travel agents etc.

Now, the important part - as you go from the bottom to the top, the learning curve gets easier, and so does the opportunity for new market players to enter. Building new chips requires billions of dollars of investments, and hence, it's harder for new players to enter the market. The most opportunities are in the top two layers. If you already know the cloud, then integrating Gen AI with your existing knowledge will increase your value immensely. If you are working in DevOps, learn MLOps; if you know K8s/Serverless, learn how you can integrate Gen AI with those; if you work in an application; integrate with managed LLM services to enhance functionality, you got the idea!

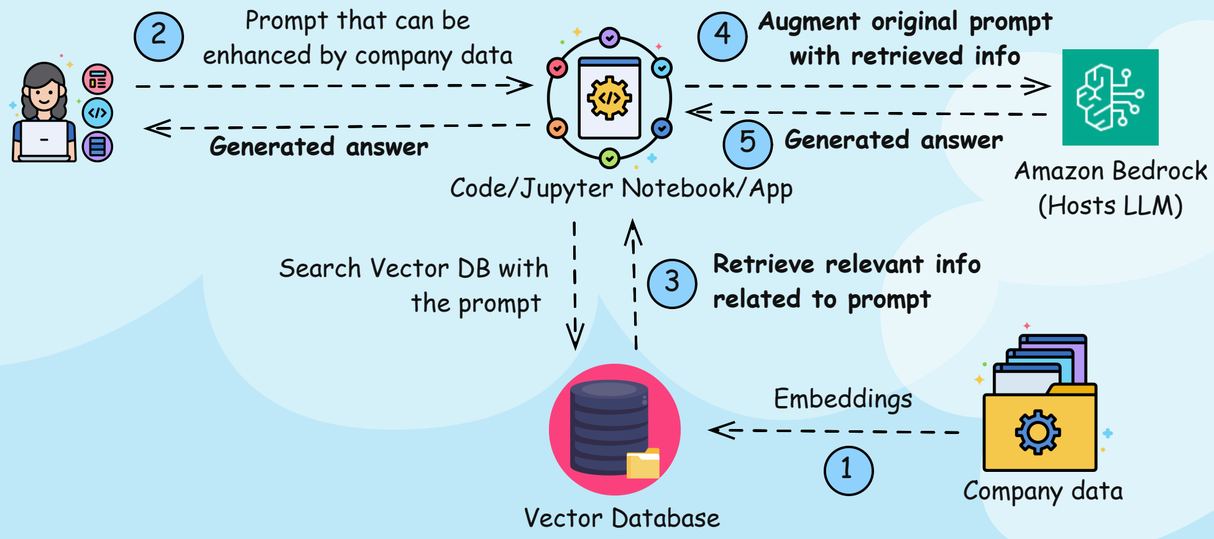
Prompt Engineering Vs RAG Vs Fine Tuning

Prompt Engineering



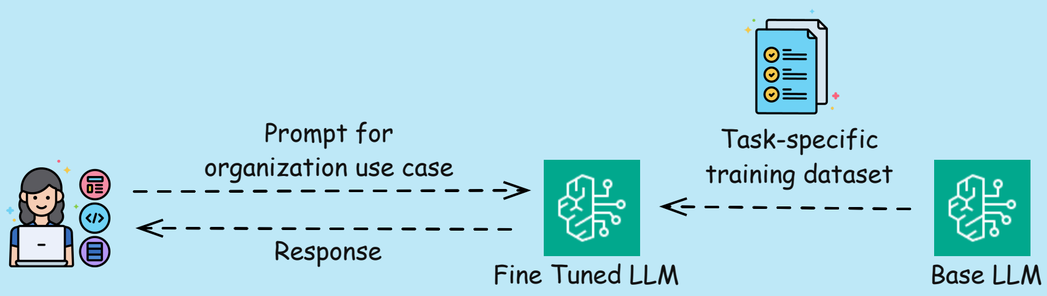
1. You send a prompt to the LLM (hosted in Amazon Bedrock in this case), and get a response which you are not satisfied with
2. You enhance the prompt, and finally come up with a prompt that gives desired better response

RAG (Retrieval Augmented Generation)



1. RAG (Retrieval Augmented Generation) is used where the response can be made better by using company specific data that the LLM does NOT have. You store relevant company data into a vector database. This is done by a process called embeddings where data is transformed into numeric vectors
2. User gives a prompt which can be made better by adding company specific info
3. A process (code/jupyter notebook/application) converts the prompt into vector and then search the vector database. Relevant info from the vector database is RETRIEVED (First Part of RAG) and returned
4. The original prompt is AUGMENTED (Second part of RAG) with this company specific info and sent to LLM
5. LLM GENERATES (Last part of RAG) the response and sends back to the user

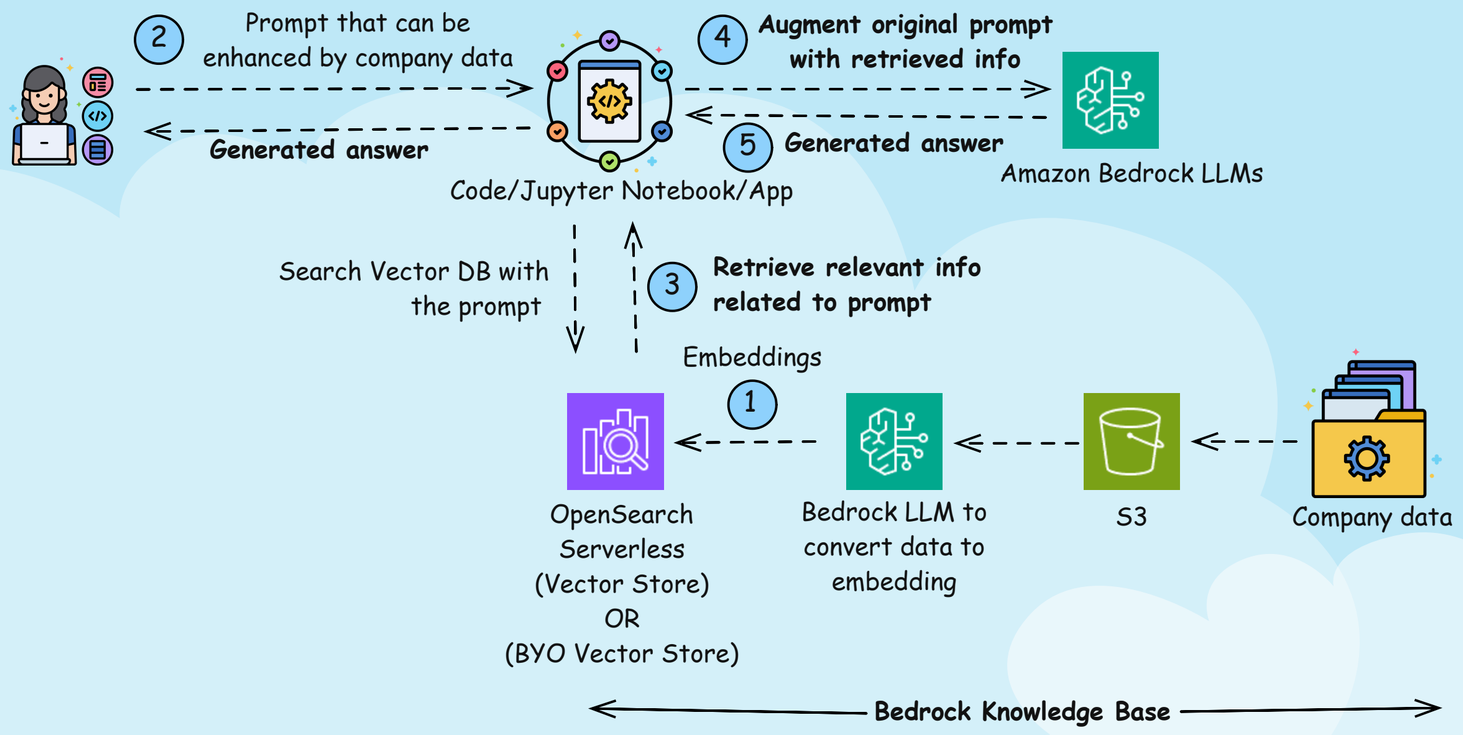
Fine Tuning



1. If you need a LLM which is very specific to your company/organization's use case that RAG can't solve, you train the base LLM with large training dataset for the tasks. The output is a fine tuned LLM.
2. User asks question related to the use case and gets answer

RAG (Retrieval Augmented Generation) with Amazon BedRock

CLOUD WITH RAJ
www.cloudwithraj.com

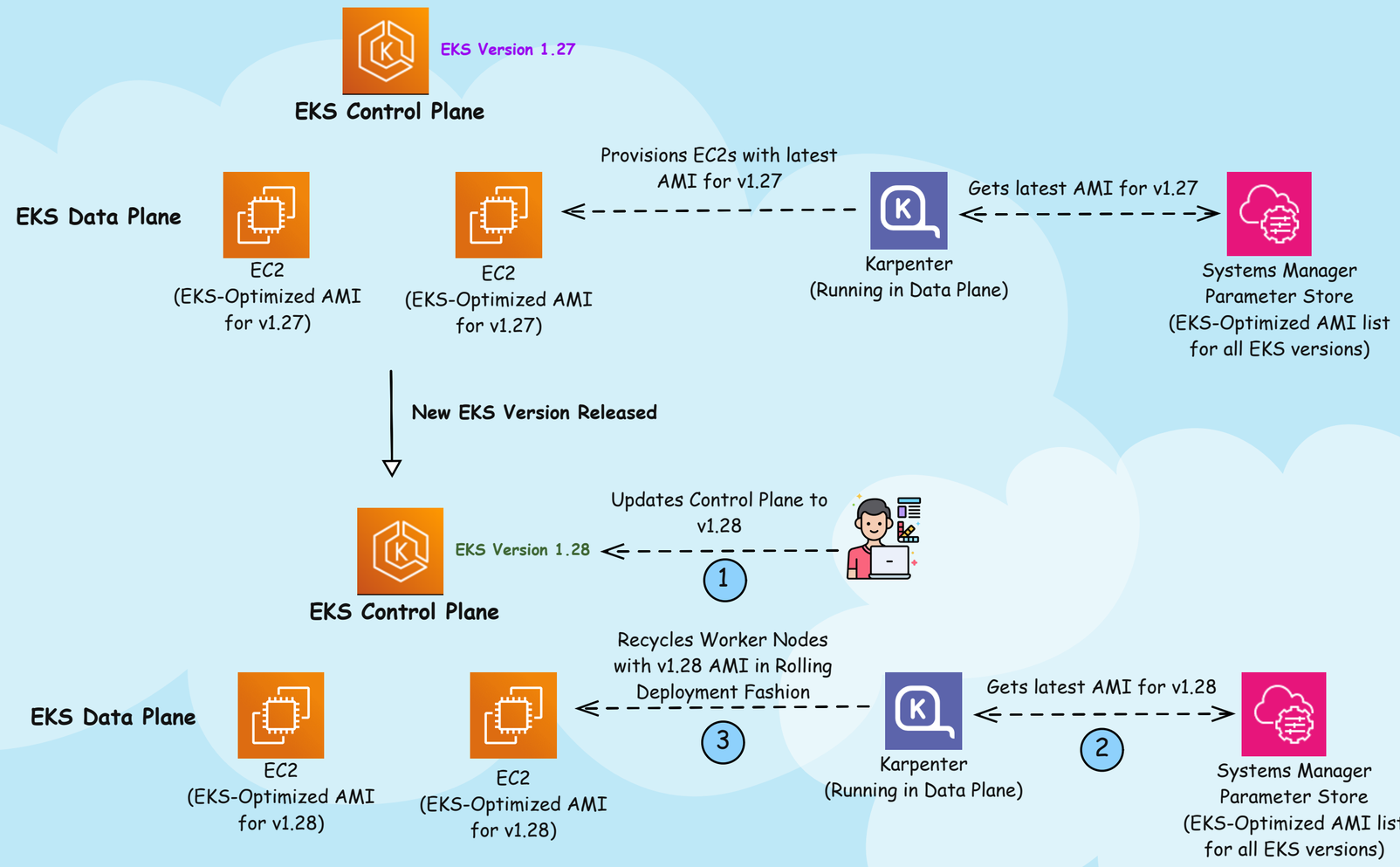


RAG (Retrieval Augmented Generation) is used where the response can be made better by using company specific data that the LLM does NOT have. Amazon BedRock makes it very easy to do RAG. Below are the steps:

1. You store relevant company data into a S3 bucket. Then from BedRock Knowledge bases, you select an emedding LLM (Amazon Titan Embed or Cohere Embed) which converts the S3 data into embeddings (vector). Knowledge Base can also create a serverless vector store for you to save those embeddings. Alternatively you can also bring your own Vector Database (OpenSearch, Aurora, Pinecone, Redis)
2. User gives a prompt which can be made better by adding company specific info
3. A process (code/jupyter notebook/application) converts the prompt into vector and then search the vector database. Relevant info from the vector database is RETRIEVED (First Part of RAG) and returned
4. The original prompt is AUGMENTED (Second part of RAG) with this company specific info and sent to another Bedrock LLM
5. BedRock LLM GENERATES (Last part of RAG) the response and sends back to the user

EKS Upgrade Simplified Using Karpenter

CLOUD WITH RAJ
www.cloudwithraj.com



EKS upgrade can be tedious. But Karpenter can automatically upgrade your Data Plane worker nodes reducing your burden. Here's how:

a. EKS Optimized AMI IDs are listed in AWS Systems Manager parameter store. CNCF project Karpenter, the next gen cluster autoscaler, periodically checks this, and reconciles with the running worker nodes to see if they are running with the latest EKS-Optimized AMI for the particular EKS version. In this case, let's assume EKS is running with v1.27.

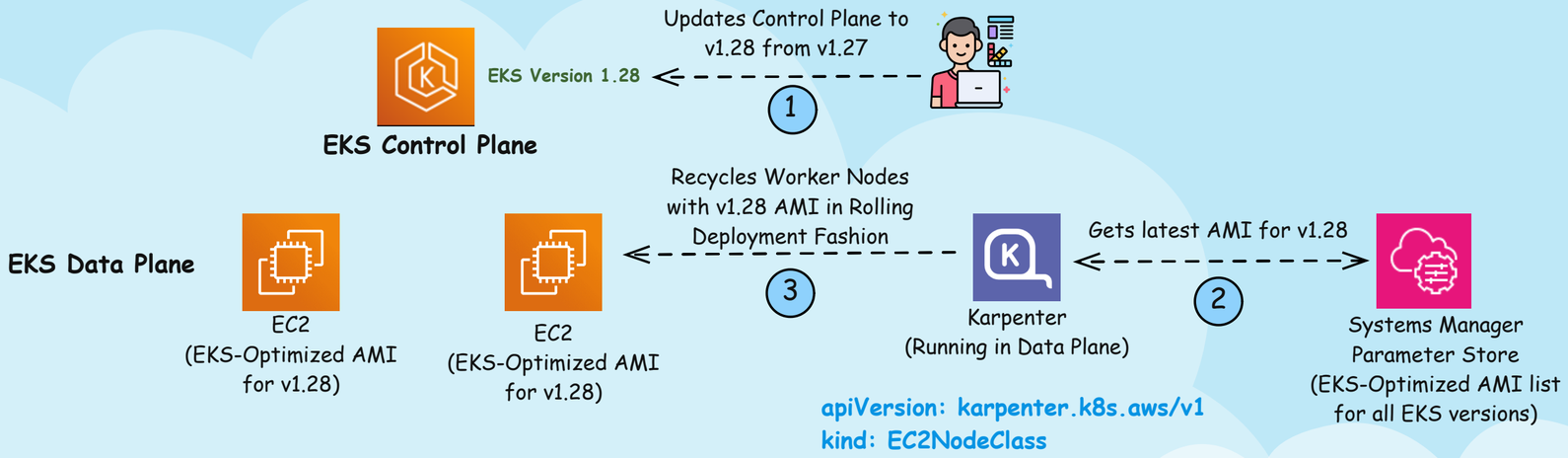
b. At a certain point, EKS releases next version 1.28. And the below workflow takes place:
1. Admin upgrades the EKS control plane to v1.28.
2. Following the previous logic, Karpenter retrieves the latest AMI for v1.28 and check if worker nodes are running with those. They are NOT, so a Karpenter Drift is triggered.
3. To fix this Drift, Karpenter automatically updates the worker nodes to v1.28 AMIs. And it does it using rolling deployment (i.e. a new node comes up, existing node cordoned and drained, then terminated). It also respects the Kubernetes eviction API parameters, such as maintaining PDB.

If you want to know the process in detail, including with custom AMIs, check out the Karpenter drift blog - https://aws.amazon.com/blogs/containers/how-to-upgrade-amazon-eks-worker-nodes-with-karpenter-drift/

EKS Upgrade Using Karpenter - Automatic and Manual

CLOUD WITH RAJ
www.cloudwithraj.com

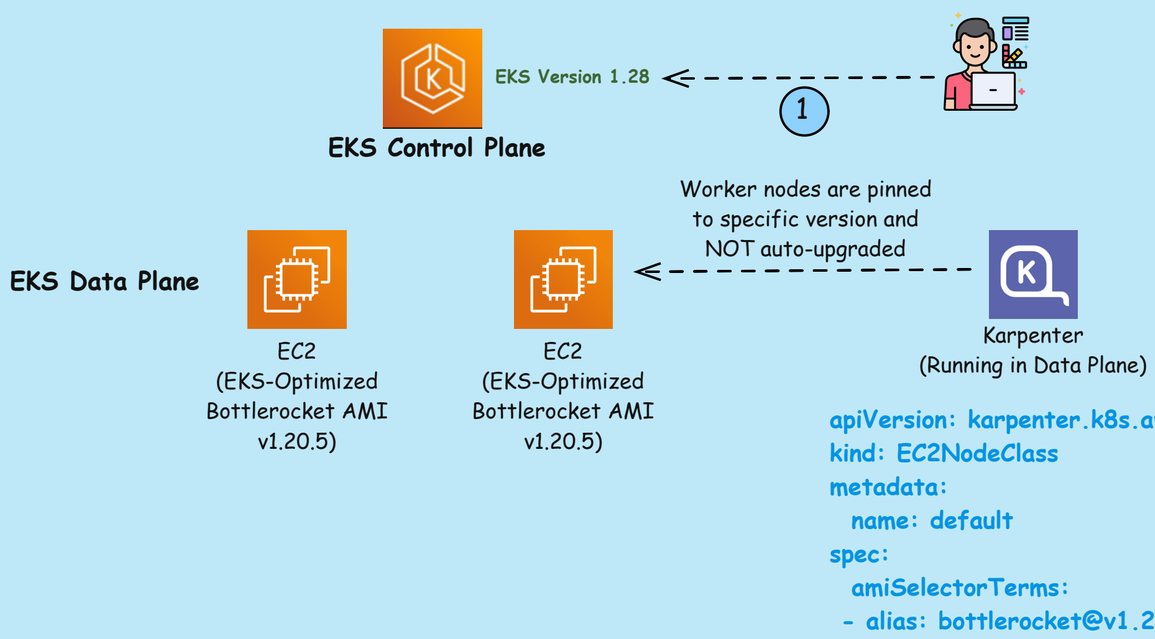
Automatic




```
apiVersion: karpenter.k8s.aws/v1
kind: EC2NodeClass
metadata:
  name: default
spec:
  amiSelectorTerms:
  - alias: bottlerocket@latest
```

The "alias: bottlerocket@latest" in EC2NodeClass ensures Karpenter will always automatically upgrade your worker nodes if AWS releases a new AMI OR you upgrade your EKS Control Plane

Manual (More Control)




```
apiVersion: karpenter.k8s.aws/v1
kind: EC2NodeClass
metadata:
  name: default
spec:
  amiSelectorTerms:
  - alias: bottlerocket@v1.20.5
```

Karpenter can automatically upgrade your K8s worker nodes. What if you don't want to - that's possible too. Here's how:

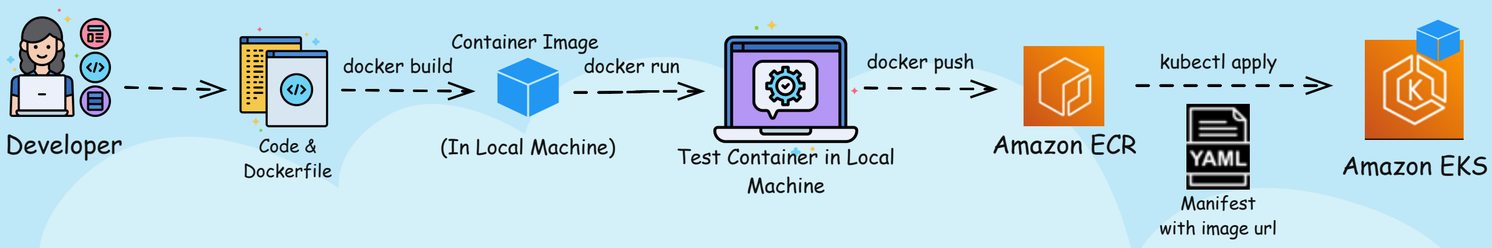
You can pin your worker nodes to a specific version of the AMI using Karpenter EC2NodeClass. For example, in the below diagram, Karpenter will provision the worker nodes with Amazon EKS BR Optimized Linux AMI for version v1.20.5.

Even after AWS releases a new AMI or you upgrade your EKS Control plane, Karpenter will NOT upgrade data plane. Remember that - EKS Data Plane can run with AMIs 3 versions behind the Control Plane. Though it is recommended to run with latest version due to security patches.

The general practice I see for critical projects with my customers is - that they will let Karpenter automatically upgrade to the latest AMI version in dev/test and pin to a specific AMI in prod. Once the newest version is tested in dev/test, production EC2NodeClass will be changed to point to this new version. Once EC2NodeClass is updated, Karpenter will upgrade the worker node AMIs in a rolling deployment fashion.

Container Lifecycle - Local to Cloud

CLOUD WITH RAJ
www.cloudwithraj.com



The fundamental container workflow from local machine to cloud is below:

1. Developer writes code, and associated Dockerfile to containerize the code in her local machine

2. She uses "Docker build" command to create the container image, in her local machine. At this point container image is saved in the local machine

3. Developer uses "Docker run" command to run the container image, and test out the code running from the container. Developer can repeat Steps 1-3, till the testing goes as per the requirements

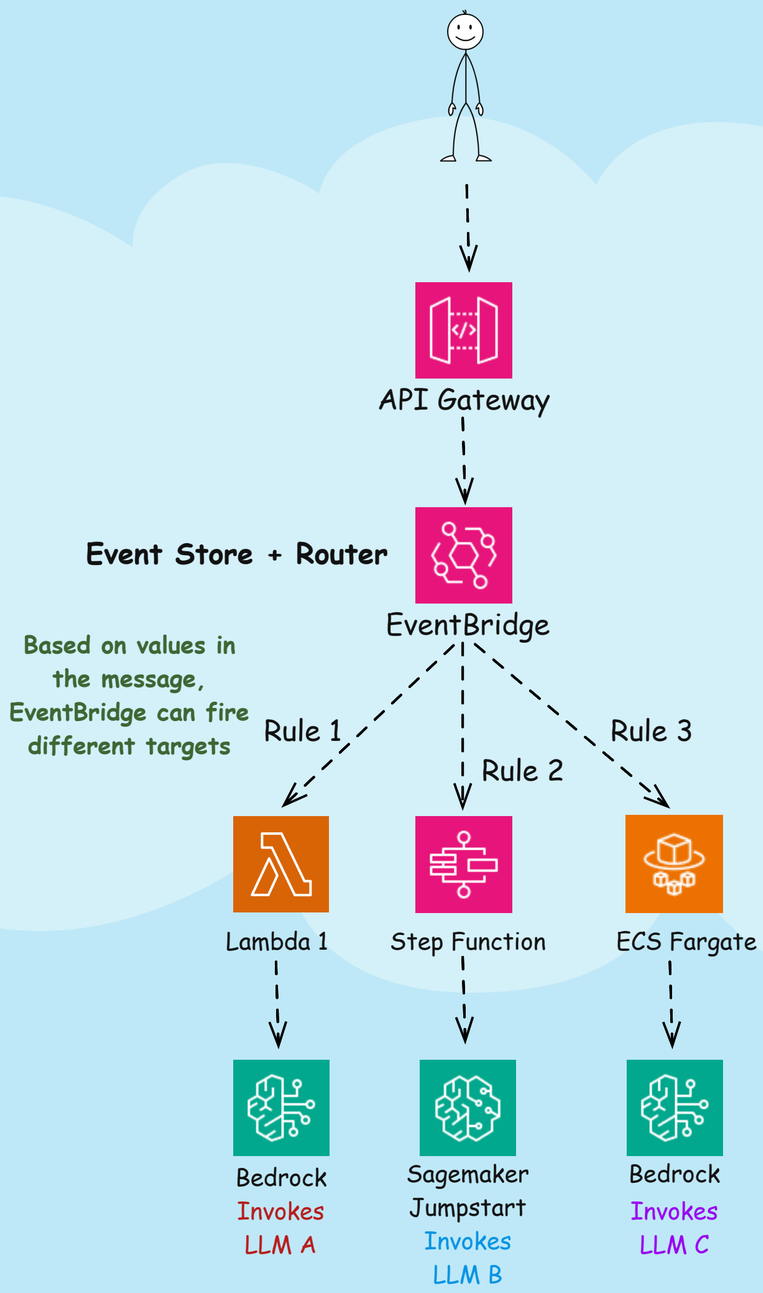
4. Next, developer runs "Docker push" command to push the container image from the local machine to a container registry. Some examples are DockerHub, or Amazon ECR.

5. Finally, using "Kubectl apply" command, an YAML manifest which has the URL of the container image from the Amazon ECR, is deployed into the running Kubernetes cluster.

Note that, this is to understand the lifecycle of the container. In real-world, after testing is done in local machine, the following steps are automated. Refer to the "Traditional CICD vs GitOps" page for that workflow

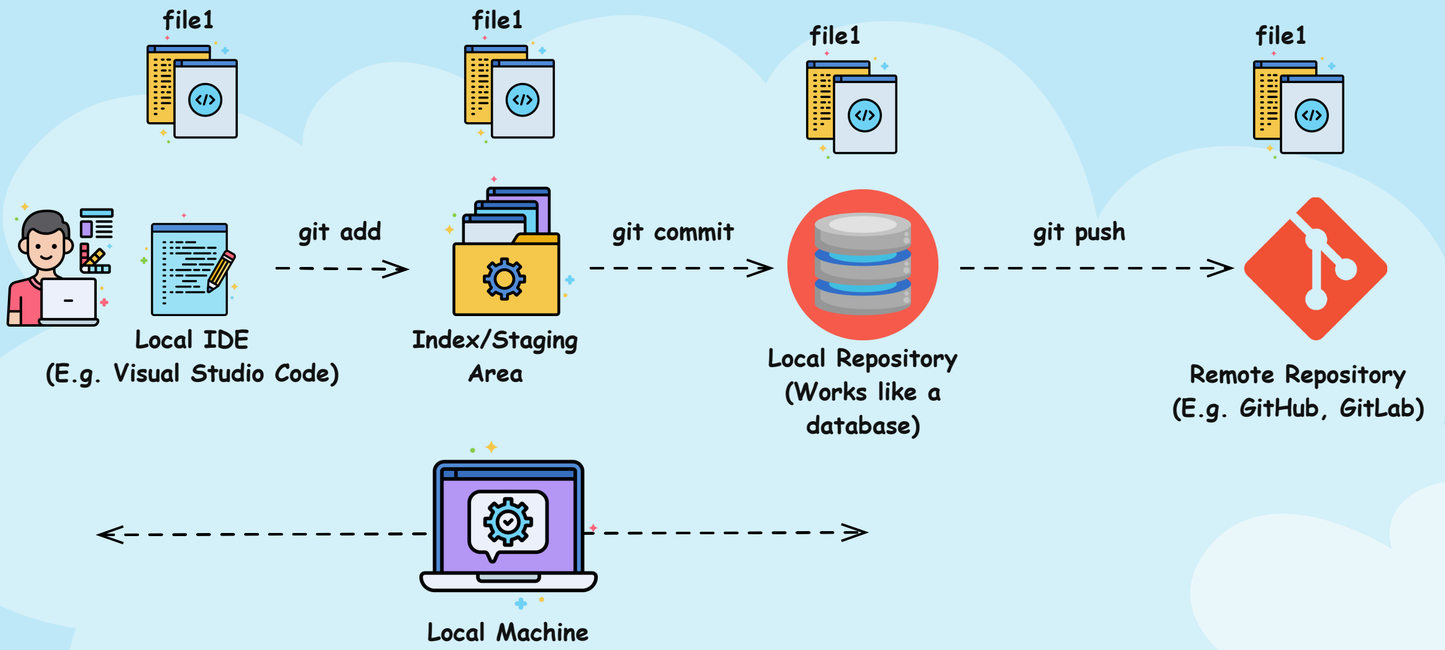
Gen AI Multi Model Invocation

CLOUD WITH RAJ
www.cloudwithraj.com



Git Workflow

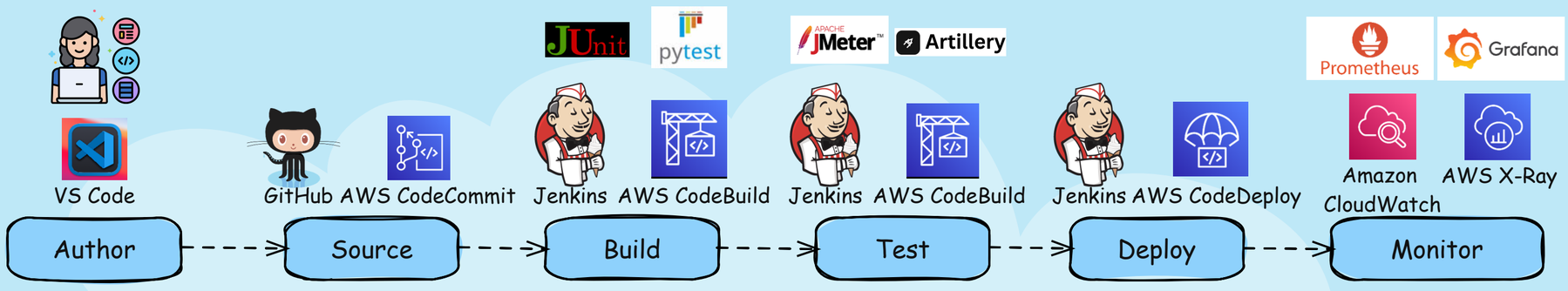
CLOUD WITH RAJ
www.cloudwithraj.com



Git and GitHub for Beginners Crash Course (Click on the YouTube icon):

DevSecOps Workflow

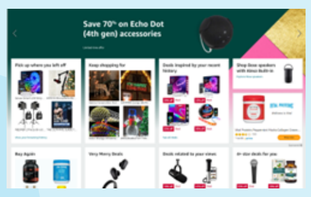
CLOUD WITH RAJ
www.cloudwithraj.com



What Happens When You Type An URL

CLOUD WITH RAJ
www.cloudwithraj.com

1 Types www.amazon.com



6 Page rendered and displayed to user

2 Checks local machine caches for IP Address of the URL

If NOT cached

DNS Resolver

3 Get IP address (LB of URL frontend)

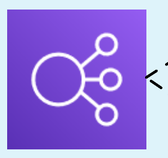
DNS root name server

Name server for .com TLD

Amazon Route 53

4 Obtained IP address

Front end pages 5



Load Balancer (LB)

EC2 Hosting Amazon.com

EC2 Hosting Amazon.com

Free video explaining steps in detail: Click

Well Architected Framework

CLOUD WITH RAJ
www.cloudwithraj.com

The AWS Well-Architected Framework describes key concepts, design principles, and architectural best practices for designing and running workloads in the cloud. By answering a few foundational questions, learn how well your architecture aligns with cloud best practices and gain guidance for making improvements. AWS Well-Architected Framework assessment questions can be answered inside your AWS account, and then a scorecard can be obtained that evaluates your application in respect to the below six pillars

Operational Excellent Pillar

The operational excellence pillar focuses on running and monitoring systems, and continually improving processes and procedures. Key topics include automating changes, responding to events, and defining standards to manage daily operations.

Security Pillar

The security pillar focuses on protecting information and systems. Key topics include confidentiality and integrity of data, managing user permissions, and establishing controls to detect security events.

Reliability Pillar

The reliability pillar focuses on workloads performing their intended functions and how to recover quickly from failure to meet demands. Key topics include distributed system design, recovery planning, and adapting to changing requirements.

Performance Efficiency Pillar

The performance efficiency pillar focuses on structured and streamlined allocation of IT and computing resources. Key topics include selecting resource types and sizes optimized for workload requirements, monitoring performance, and maintaining efficiency as business needs evolve.

Cost Optimization Pillar

The cost optimization pillar focuses on avoiding unnecessary costs. Key topics include understanding spending over time and controlling fund allocation, selecting resources of the right type and quantity, and scaling to meet business needs without overspending.

Sustainability Pillar

The sustainability pillar focuses on minimizing the environmental impacts of running cloud workloads. Key topics include a shared responsibility model for sustainability, understanding impact, and maximizing utilization to minimize required resources and reduce downstream impacts.

AWS Migration Tools

CLOUD WITH RAJ
www.cloudwithraj.com